Task 1 Team Guide: Who Does What

Dataset understanding + Chapter 2 probability | 5 members | Smart Buildings (NeurIPS 2026 kit)

1. The plan in one page

Your project has one deliverable notebook (Google Colab) and one slide deck (max 15 slides, max 8 minutes, exported to PDF). The work is split so that every member owns a clear block of notebook sections **and** a block of slides, and every member must be able to explain the whole task during Q&A.

Member	Role	Notebook sections	Slides
1	Data Lead	Setup, dataset understanding, preprocessing, event columns	1-3
2	Probability A	Sample space, axioms, complement, counting, addition rule, rejected-seed analysis	4-5
3	Probability B	Conditional, multiplication, total probability, Bayes	6-7
4	Independence + Validation	Independence tests, 3-way validation, Monte Carlo, bootstrap, sensitivity	8-9
5	Insights + Presentation	Intro text cell, final charts, insights, limitations, competition relevance, deck, final QA	10-12 + assembly

Dataset facts everyone must know (verified from your files)

File	Shape	One row =	Link key
buildings_200_features.csv	200 x 54	one generated building	cfg__seed
rejected_seeds.csv	82 x 2	one seed the generator refused (with text reason)	seed
feature_catalog.csv	133 x 4	one feature definition (data dictionary; 54 in CSV, 79 simulation-only)	column

- Seeds 100000-100281 = **282** seeds tried. 200 accepted + 82 rejected, no gaps, no overlap. This is your natural sample space.
- No missing values, no duplicate rows. About 17 columns are constant (carry no information); several are redundant (n_zones = n_vav_boxes in all rows; n_ahus = num_ahus + ahus_added in all rows).
- n_zones is right-skewed: min 21, median 64, max 305. Floors: 2 floors = 74, 3 = 63, 4 = 63 buildings.
- Probabilities describe the **generator**, not real buildings. Say this clearly in limitations.

Working rules (read before writing any code)

1. **One master notebook, owned by Member 1.** Everyone else develops in their *own copy* (File > Save a copy in Drive), named T1_M2_draft, T1_M3_draft, etc. Do not edit the master at the same time - Colab merges badly.
2. **The contract:** Member 1 publishes the cleaned DataFrame `df` and the event columns on **Day 1** (list in Section 2). Nobody renames them afterwards.
3. **Never modify raw data.** Keep df_raw, rej_raw, cat_raw untouched; always use .copy().
4. **Every application uses the same 5-part template:** Question; Events and assumptions; Rule and why it fits; Calculation and result; Interpretation. Put the first 3 parts in a text cell *before* the code and the last 2 after.
5. Use a fixed random seed (42) everywhere so reruns give identical numbers.
6. Final check by Member 5: Runtime > Restart and run all. If anything fails, the owner fixes it the same day.

Timeline

Day	Goal	Who
1	Master notebook skeleton, data loaded, df and event columns published; everyone reads the assignment PDF and the report	M1 (all read)
2	Draft applications and first results in personal copies	M2, M3, M4
3	Validation table filled; charts drafted; text cells written	M4, M5 (M2, M3 finish)
4	Merge into master (M1); Restart and run all; slides built	M1, M5
5	Rehearsal with timer; cross-Q&A drill; submit PDF + Colab link	All

Member 1

Data Lead
Owns: Notebook Sections 1-4 | Slides 1-3 | Master notebook owner

Goal: deliver a clean, documented, reproducible dataset foundation that all later tasks reuse. This is graded as "understanding of the dataset" and "quality of data preparation".

Steps

1. **Set up.** Create a shared Drive folder, upload the 3 CSVs, create the master Colab notebook, share it as Editor with all 5 members and set link sharing so the instructor can view. Create 8 empty section headings (0 Intro, 1 Setup, 2 Understanding, 3 Preprocessing, 4 Events, 5 Applications, 6 Validation, 7 Insights) and tag each with its owner.
2. **Load data** (code below). Print shapes. Confirm 200x54, 82x2, 133x4.
3. **Describe files:** a table of file, rows, columns, what a row means, link key. Explain that rejected_seeds + buildings = the 282-seed universe and that feature_catalog documents columns (54 in the CSV; 79 observation features exist only during simulation).
4. **Variable dictionary:** for all 54 columns give name, group (meta/cfg/derived/obs), meaning, type. Types to use: numeric-discrete (num_floors, num_ahus, n_zones), numeric-continuous (width, composite_coverage, material properties), categorical (building_shape, composite_style), text/identifier (name, sha256, generator_output_files, floor_plan_shape), date/time (start_timestamp). Take meanings from feature_catalog.csv, then add your own plain-English column.
5. **Show a sample:** df_raw.sample(5, random_state=42) on 8-10 representative columns (the full 54 are unreadable on a slide).
6. **Quality checks** - run and report each: missing values total (0); duplicated rows (0); seed unique; sha256 unique; n_zones == n_vav_boxes (True); n_ahus == num_ahus + ahus_added (True); accepted and rejected seed sets are disjoint and together equal 100000-100281.
7. **Preprocess with a log** (code below). Report rows before/after every step. Expected: 200 -> 200 (nothing removed, because no missing or duplicate rows), columns 54 -> about 37 after dropping constants in the analysis frame only.
8. **Parse rejected seeds** into n_small_zones and smallest_m2 with regex; build the 282-row seed table with an accepted flag.
9. **Create the event columns** (Section 2 of this PDF) and post the column list in the group chat.
10. **Sensitivity helper:** create df_trim = df without the top 5% of n_zones, for Member 4.
11. **Write the limitations paragraph** for the data: generator output not real-world, no outcome variable, constants/redundancy, derived observation lengths not measured per building, weather preset fixed.
12. **Merge** everyone's cells into the master on Day 4 and keep section order and numbering consistent.

Starter code

```
import pandas as pd, numpy as np, re
DATA_DIR = "/content/drive/MyDrive/stats_project/"        # change once, everyone uses it
df_raw  = pd.read_csv(DATA_DIR + "buildings_200_features.csv")
rej_raw = pd.read_csv(DATA_DIR + "rejected_seeds.csv")
cat_raw = pd.read_csv(DATA_DIR + "feature_catalog.csv")
print(df_raw.shape, rej_raw.shape, cat_raw.shape)

log = []
def step(name, d):
    log.append((name, len(d), d.shape[1])); print(f"{name}: {len(d)} rows x {d.shape[1]} cols")

step("raw buildings", df_raw)
df = df_raw.copy()
df.columns = [c.split("__", 1)[1] for c in df.columns]    # shorter names: n_zones, num_floors...
const = [c for c in df.columns if df[c].nunique() == 1]
df = df.drop(columns=const)                                # analysis frame only; raw is untouched
step(f"drop {len(const)} constant columns", df)

rej = rej_raw.copy()
rej["n_small_zones"] = rej.reason.str.extract(r"produces (\d+) zone")[0].astype(int)
rej["smallest_m2"]   = rej.reason.str.extract(r"smallest ([\d.]+) m2")[0].astype(float)
seeds = pd.DataFrame({"seed": range(100000, 100282)})
seeds["accepted"] = seeds.seed.isin(df.seed)
assert seeds.accepted.sum() == 200 and (~seeds.accepted).sum() == 82
```

Event columns you must publish (the contract)

```
# NOTE: after dropping constants, check the exact names with df.columns (floor_plan.num_floors etc.)
med = df.n_zones.median()                                  # 64
df["A_large"]     = df.n_zones > med                       # 98 buildings
df["B_extra_ahu"] = df.ahus_added_beyond_config > 0        # 146 buildings
df["F2"], df["F3"], df["F4"] = [df["floor_plan.num_floors"] == k for k in (2, 3, 4)]
df["O2"] = df["simulation.occupants_per_zone"] == 2        # 108 buildings
```

Document in text cells (why + effect on probabilities)

- Nothing removed: so P(...) is computed on all 200 accepted buildings. Mention these are *accepted* buildings only, so all results are conditional on acceptance by the generator.
- Constants dropped from the analysis frame: they cannot form a non-trivial event; no probability changes.
- Outliers (up to 305 zones) kept: they are valid generator output. Sensitivity (Member 4) shows how the main probabilities move if the top 5% is removed.

Common pitfalls

- Dropping the median-tie rows by accident: A_large uses strictly greater than 64 (98 rows); 6 buildings have exactly 64.
- Overwriting df_raw. Always copy.
- Forgetting to report rows before/after - the assignment requires it.

Member 2

Probability A - foundations and counting
Owns: Notebook Section 5.1-5.4 | Slides 4-5

Goal: apply sample spaces and events, the axioms, the complement rule, counting techniques and the addition rule - each with a meaningful question, not just a formula.

Application 1 - Sample space and events (the generator run)

1. Question: What is the chance that a seed the generator tries is rejected?
2. Sample space S = the 282 seeds (100000-100281). Event R = rejected (82 seeds). Assumption: seeds behave like equally likely outcomes (counting-based probability).
3. Rule: P(R) = |R| / |S|.
4. Calculation: 82/282 = **0.2908**. Interpretation: about 29% of seeds are wasted; to get N usable buildings you need about N / 0.709 seeds.

```
S_size = len(seeds); R = (~seeds.accepted).sum()
print("P(R) =", R / S_size)            # 0.2908
```

Application 2 - Axioms and complement rule

1. Question: Is the empirical distribution of num_floors a valid probability model, and what is P(no extra AHU)?
2. Show: every P(k) >= 0; P(S) = sum of P(2)+P(3)+P(4) = 1 (0.37 + 0.315 + 0.315); additivity for disjoint events (F2 and F3 cannot both happen).
3. Complement: P(not B) = 1 - P(B) = 1 - 0.73 = **0.27** (54/200). Compare with the direct count. Interpretation: only about 1 in 4 buildings stays within its configured AHU count.

```
pf = df["floor_plan.num_floors"].value_counts(normalize=True).sort_index()
print(pf, pf.sum())                              # 0.37, 0.315, 0.315 -> 1.0
print(1 - df.B_extra_ahu.mean(), (~df.B_extra_ahu).mean())
```

Application 3 - Counting techniques

- **Combinations (without replacement):** a reviewer picks 3 buildings at random. P(all 3 needed extra AHUs) = C(146,3) / C(200,3) = **0.3869**. Compare with the 'with replacement' approximation 0.73^3 = 0.389 and explain the small gap (finite population).
- **Permutations:** number of ways to choose an ordered (train, validation, test) triple of buildings = 200P3 = 200*199*198 = 7,880,400. Contrast with C(200,3) to show when order matters.
- **Configuration space (multiplication principle):** 3 floor options x 5 AHU options (3-7) x 2 occupant options = 30 configuration cells. Count how many of the 30 cells actually appear in the 200 buildings and show the empty cells, if any.
- **Train/test split:** number of possible 140/60 splits = C(200,140) (print it with len(str(...)) digits - it is astronomically large; this is why a fixed seed matters).

```
from math import comb, perm
print(comb(146, 3) / comb(200, 3), 0.73 ** 3)       # 0.3869 vs 0.3890
print(perm(200, 3), comb(200, 3))
cells = df.groupby(["floor_plan.num_floors", "ahu.num_ahus",
                    "simulation.occupants_per_zone"]).size()
print(len(cells), "of 30 cells observed")
```

Application 4 - Addition rule

- General rule: P(A or B) = P(A) + P(B) - P(A and B) = 0.49 + 0.73 - 0.485 = **0.735** (large OR extra AHU). Verify with (A | B).mean().

- Special case (mutually exclusive): P(2 floors or 4 floors) = 0.37 + 0.315 = **0.685**. Explain why subtracting the intersection is unnecessary here.

Application 5 - Rejected seeds as a second probability model

- Define events on the 82 rejected seeds: E1 = exactly 1 undersized zone; E2 = smallest zone under 5 m2; E3 = 3 or more undersized zones.
- Report P(E1 | R), P(E2 | R), P(E3 | R) and the distribution of n_small_zones (mean about 1.59, max 7). Interpretation: most failures are a single tiny zone, so a minor floor-plan fix would rescue most seeds.

Deliverables checklist

- 4-5 applications, each with the 5-part template.
- One chart: bar of P(num_floors) and one of n_small_zones.
- Hand numbers to Member 4 for validation (a dict of rule-based results).

Common pitfalls

- Using n=200 when the question is about seeds (282).
- Calling the sample 'random' - say 'treated as equally likely draws from the generator output'.

Member 3

Probability B - conditional reasoning
Owns: Notebook Section 5.5-5.8 | Slides 6-7

Goal: show how knowing one fact changes the probability of another, and use total probability and Bayes' theorem to reverse the direction of reasoning.

Application 6 - Conditional probability

1. Question: Are large buildings more likely to need extra AHUs than small ones?
2. Events: A = n_zones > 64; B = real AHUs exceed the configured number. Assumption: finite population, so P(B|A) = count(A and B) / count(A).
3. Calculation: P(B|A) = 97/98 = **0.990**; P(B | not A) = 49/102 = **0.480**; P(B) = 0.730.
4. Interpretation: nearly every large building needs extra AHUs because the generator adds handlers when rooms exceed the cap of 10 rooms per AHU (see the report). Size is a strong predictor.

```
def Pc(a, b):                       # P(a | b)
    return (a & b).sum() / b.sum()
A, Bx = df.A_large, df.B_extra_ahu
print(Pc(Bx, A), Pc(Bx, ~A), Bx.mean())      # 0.9898, 0.4804, 0.73
```

Application 7 - Multiplication rule

- P(A and B) = P(A) * P(B|A) = 0.49 * 0.9898 = **0.485**. Verify with (A & B).mean() = 97/200.
- Extend with the chain rule: P(F4 and A and B) = P(F4) * P(A|F4) * P(B|F4 and A). Compute each factor from the data and compare with the direct count of rows.

Application 8 - Total probability

1. Question: Can we obtain P(extra AHU) from the floor counts alone?
2. Partition: F2, F3, F4 are mutually exclusive and cover all buildings (74+63+63 = 200).
3. Rule: P(B) = sum over k of P(B|Fk) * P(Fk).
4. Calculation: (39/74)(0.37) + (49/63)(0.315) + (58/63)(0.315) = 0.195 + 0.245 + 0.290 = **0.730**. This should match the direct 146/200.
5. Interpretation: more floors means more rooms, so the share needing extra AHUs rises: 0.53 (2 floors), 0.78 (3), 0.92 (4).

Application 9 - Bayes' theorem

1. Question: A building is known to have needed extra AHUs. What is the chance it has 4 floors?
2. Rule: P(F4|B) = P(B|F4) * P(F4) / P(B), where the denominator is the total-probability result above.
3. Calculation: (58/63)(0.315) / 0.73 = **0.397**, versus the prior P(F4) = 0.315. Verify with 58/146.
4. Second Bayes question: P(A|B) = P(B|A)P(A)/P(B) = 0.9898*0.49/0.73 = **0.664** (97/146): of buildings that needed extra AHUs, about two thirds are large.
5. Interpretation: observing the extra-AHU 'evidence' shifts belief toward more floors and larger size. Explain prior vs posterior in plain words.

```
pF = {k: df[f"F{k}"].mean() for k in (2, 3, 4)}
pB_F = {k: Pc(Bx, df[f"F{k}"]) for k in (2, 3, 4)}
pB_total = sum(pB_F[k] * pF[k] for k in (2, 3, 4))
post_F4 = pB_F[4] * pF[4] / pB_total
print(pB_total, Bx.mean(), post_F4, 58 / 146)
```

Deliverables checklist

- A probability tree (Visualizer or matplotlib) for floors -> extra AHU.

- A small table prior vs posterior for floors 2/3/4.
- Hand all results to Member 4 in a dict (rule-based values).

Common pitfalls

- Mixing up P(B|A) and P(A|B) - write both in words next to every number.
- Using a partition that does not sum to 1 for total probability (check the counts add to 200).
- Do not say 'A causes B'. These are associations in generated data.

Member 4

Independence + Validation
Owns: Notebook Section 5.9 and Section 6 | Slides 8-9

Goal: test which events are independent, and prove the main results are correct by computing them in independent ways. This section earns the 'reliability and validation' marks.

Part A - Independence (Application 10)

1. Definition to state: A and B are independent if P(A and B) = P(A) P(B), equivalently P(B|A) = P(B).
2. Write one function that prints P(A), P(B), P(A and B), P(A)P(B), the difference, and the lift = P(A and B)/(P(A)P(B)). Lift near 1 suggests independence.
3. Tolerance: with n = 200 even independent events differ by noise. Use SE = sqrt(p(1-p)/n) for the joint probability and flag a pair as 'consistent with independence' when |difference| is within about 2 SE. State this is a judgement rule; formal tests come in a later chapter.
4. Pairs to test: (1) A_large vs B_extra_ahu: P(A)P(B) = 0.358 vs P(A and B) = 0.485 -> **dependent**. (2) F4 vs B. (3) O2 vs B. (4) F4 vs O2. (5) F4 vs (num_ahus == 7). (6) O2 vs A_large. I have not computed pairs 2-6; run them.
5. **Story to build:** the generator draws configuration parameters (floors, num_ahus, occupants) independently, so config-vs-config pairs should look independent. Derived outcomes (extra AHUs, zone count) depend on config. If your results show this, you have validated the generator itself. If a config pair looks dependent, investigate (small sample? built-in coupling?).

```
def indep(a, b, name):
    pa, pb, pab = a.mean(), b.mean(), (a & b).mean()
    se = (pa * pb * (1 - pa * pb) / len(a)) ** 0.5
    print(f"{name}: P(A)P(B)={pa*pb:.3f}  P(A&B)={pab:.3f}  lift={pab/(pa*pb):.2f}  "
          f"diff={pab-pa*pb:+.3f}  2SE={2*se:.3f}")
indep(df.A_large, df.B_extra_ahu, "large vs extra AHU")
```

Part B - Three-way validation table (Section 6)

For each major result from Members 2 and 3 compute three values and compare:

Route	How	Expected relation
1. Rule-based	Formula with marginals/conditionals (total probability, Bayes, multiplication, addition)	reference
2. Direct count	Boolean filter on rows, e.g. (A & B).sum()/len(df)	Identical to route 1 (same finite population). If not, there is a bug.
3. Monte Carlo	Resample 100,000 rows with replacement (seed 42) and recompute	Differs by sampling error about sqrt(p(1-p)/100000), roughly 0.0014 for p = 0.5

```
rng = np.random.default_rng(42)
sim = df.loc[rng.choice(df.index, 100_000)]
rows = []
for name, rule, direct, mc in [
    ("P(B)",        pB_total,  df.B_extra_ahu.mean(),      sim.B_extra_ahu.mean()),
    ("P(B|A)",      (A & Bx).mean() / A.mean(), Pc(Bx, A), Pc(sim.B_extra_ahu, sim.A_large)),
    ("P(F4|B)",     post_F4,   Pc(df.F4, Bx),              Pc(sim.F4, sim.B_extra_ahu))]:
    rows.append((name, rule, direct, mc, abs(rule-direct), abs(direct-mc)))
val = pd.DataFrame(rows, columns=["event","rule","direct","monte_carlo","|rule-direct|","|direct-MC|"])
```

- Also validate the counting result: simulate drawing 3 buildings without replacement 100,000 times and compare with C(146,3)/C(200,3) = 0.3869.
- **Bootstrap CI:** resample 200 rows 2,000 times, recompute P(B|A) each time, take the 2.5th and 97.5th percentiles. Interpret the width as uncertainty from having only 200 buildings.

Part C - Preprocessing sensitivity

- Recompute 4-5 key probabilities on Member 1's df_trim (top 5% of zones removed) and show the change in a table. Explain which results are robust and which move, and why.
- Explicitly distinguish: **exact agreement** (rule vs direct), **sampling differences** (Monte Carlo, bootstrap), and **modeling differences** (with vs without replacement; treating generator output as random draws).

Deliverables checklist

- Independence table with verdict per pair and a bar chart of lift values.
- Validation table (also as an image for the slides).
- Bootstrap histogram for P(B|A).
- Sensitivity table.

Common pitfalls

- Declaring independence just because numbers are 'close' - always compare to SE.
- Forgetting the fixed seed; results must be reproducible.
- Hard-coding numbers instead of pulling live variables from Members 2 and 3.

Member 5

Insights + Presentation + Final QA
Owns: Notebook Section 0 and Section 7 | Slides 10-12 and deck assembly

Goal: turn the numbers into a story the instructor can follow in 8 minutes, and make sure the notebook meets every formatting requirement.

Notebook responsibilities

1. **Section 0 intro text cell** (required first cell): the problem, the objectives and the analysis steps. Suggested problem: *How likely is a generated smart building to be complex (needing extra air handlers), and which design factors predict it? This matters for building fair train/test sets and curricula in the NeurIPS 2026 Smart Buildings control challenge.*
2. **Charts** (agree styling: one palette, titles, axis labels, readable fonts): (a) histogram of zones per building with median line and threshold; (b) P(extra AHU) by floors; (c) P(B|A) vs P(B|not A) vs P(B); (d) joint probability heatmap floors x extra AHU; (e) probability tree (from M3); (f) rejected seeds: n_small_zones histogram; (g) lift per pair (from M4); (h) validation table.
3. **Section 7 insights:** 4-6 findings, each with number + meaning. Examples to confirm with real output: size nearly guarantees extra AHUs (0.99 vs 0.48); more floors raises the extra-AHU rate (about 0.53 / 0.78 / 0.92); about 29% of seeds are rejected, mostly for a single tiny zone; configuration parameters look independent of each other while derived outcomes depend on them (if M4 confirms).
4. **Assumptions and limitations:** generator output, not real buildings; results conditional on accepted seeds; n = 200; median threshold is arbitrary (show another threshold if time allows); no outcome variable (energy, comfort); constants and redundant columns; observation length is derived by formula, not measured in every building.
5. **Competition relevance:** (1) Probability a random building is 'complex' guides stratified train/test splits; (2) observation vector length ranges from 229 to 2,353 values, so complex buildings are much harder for a controller - a curriculum from easy to hard is justified; (3) rejection rate tells organizers how many seeds to generate for N usable buildings; (4) the dependence structure shows which parameters to vary to cover the space.
6. **Final QA:** Runtime > Restart and run all; check every code section is preceded by a text cell; row-before/after counts appear; no stray errors; sharing set so the instructor can open it.

Slide deck (max 15 slides, aim 12, 8 minutes = about 40 s per slide)

#	Content	Owner of content
1	Dataset structure: 3 files, rows/columns, variable types (numeric, categorical, text, date), what a row is, how files link. **Required.**	M1
2	Problem, objectives, why it matters for the competition	M5
3	Preprocessing: steps with rows before/after; what was kept and why	M1
4	Sample space, events, axioms, complement	M2
5	Counting techniques and addition rule	M2
6	Conditional probability and multiplication rule	M3
7	Total probability and Bayes (with the tree)	M3
8	Independence results (lift chart)	M4
9	Validation table and what the differences mean	M4
10	Key insights (2 strongest charts)	M5
11	Assumptions and limitations	M5
12	Use in the competition and conclusion	M5

- One idea per slide, big numbers, max 3 bullets, charts instead of tables where possible.

- Only one person presents. Pick the presenter now; later tasks must use a different person until everyone has presented.
- Rehearse with a timer on Day 5; cut slides rather than speed up.

2. Final checklist before submitting

Requirement from the assignment	Owner	Done
First cell = text cell with problem, objectives, steps	M5	[ ]
Files, rows, columns, variable types and meanings, row definition, file relations	M1	[ ]
Sample of data shown	M1	[ ]
Missing values, duplicates, limitations discussed	M1	[ ]
Every preprocessing step documented with rows before/after and justification	M1	[ ]
Effect of preprocessing on probabilities explained (sensitivity)	M4	[ ]
Sample spaces/events, counting, axioms, addition, multiplication, conditional, total probability, independence, Bayes all applied	M2, M3, M4	[ ]
Each application has question, events/assumptions, rule, calculation, interpretation	M2, M3, M4	[ ]
Where a concept does not apply directly, it is explained and an alternative used	M2, M5	[ ]
Rule-based vs direct vs independent calculation; differences explained	M4	[ ]
Probability tables and charts; patterns explained	M5	[ ]
Assumptions, limitations, competition usefulness	M5	[ ]
Notebook reruns top to bottom with no errors; Python only	M1, M5	[ ]
Slides PDF (max 15 slides, under 8 min), slide 1 describes dataset	M5	[ ]
Colab link opens for the instructor	M1	[ ]

3. Q&A preparation (everyone should answer these)

1. Why did you choose the median (64 zones) as the threshold, and what happens with another threshold?
2. Why is P(B|A) so different from P(B)? Which rule shows it?
3. Why do the rule-based and direct counts match exactly, but Monte Carlo differs slightly?
4. What is the difference between C(200,3) and 200P3, and when would you use each?
5. Why can't you say that size causes extra AHUs?
6. No rows were removed - so what did preprocessing change?
7. Why do results apply to 'accepted' buildings only, and how would including rejected seeds change the conclusion?
8. How will this dataset support the next tasks (distributions, estimation, hypothesis tests, regression)? Be ready to mention the lack of an outcome variable and your plan for it.

Honest risk to decide now: the dataset has no outcome variable (energy use, comfort). Later chapters can still use n_zones, extra-AHU status and the observation length as targets, but agree early whether you will keep this dataset or add simulation rollout results, because the assignment allows changing the dataset only in Task 1.